

TanaClean 2035

Système intelligent d'optimisation de la collecte des déchets
à Antananarivo, Madagascar

Electron

Node.js

Express

HTMX

SQLite

Dijkstra

Auteur	ANDRIAMAMONJISOA Amboara
NIE	SE20240277
Formation	Licence Informatique
Année	2025 – 2026
Langage	JavaScript (Node.js)
Base de données	SQLite (dev) → MySQL (prod)
Version doc.	1.0 — Mai 2026

TanaClean 2035

Système intelligent d'optimisation de la collecte des déchets
à Antananarivo, Madagascar

Electron

Node.js

Express

HTMX

SQLite

Dijkstra

Table des matières

01 Présentation du projet

- Contexte et objectifs
- Périmètre fonctionnel

02 Architecture du système

- Couches applicatives
- Flux de démarrage Electron
- Arborescence

03 Stack technique

- Technologies utilisées
- Justifications des choix

- Commandes dev

04 Base de données

- Schéma relationnel (9 tables)
- Graphe des quartiers
- Cycles de statut

05 Profils utilisateurs

- Administrateur
- Éboueur
- Citoyen

06 Flux métier

- Authentification
- Cycle réclamation
- Planification tournée

07 Opérations par profil

- 18 opérations admin
- 5 opérations éboueur
- 5 opérations citoyen

08 Algorithmes

- Dijkstra — $O((V+E) \log V)$
- Glouton — $O(n \log n)$
- Union-Find — $O(\alpha)$

09 Routes HTTP et API

- Routes publiques
- Routes admin / worker / citizen

10 Sécurité et déploiement

- Mesures de sécurité
- Variables d'environnement
- Build & distribution

11 Version desktop & sécurité avancée

- Séparation des serveurs (3131 web / 3132 desktop)
- 5 couches de protection admin
- Fix pagination paramètres

01 Présentation du projet

TanaClean 2035 est une application desktop développée pour moderniser et optimiser la collecte des déchets dans la ville d'Antananarivo (Madagascar). Face à l'extension géographique de la ville et à l'augmentation de sa population, la gestion manuelle des tournées de collecte entraîne des inefficacités : trajets non optimisés, retards non communiqués, réclamations sans suivi. TanaClean 2035 répond à ces problèmes en centralisant dans une seule application desktop la gestion des ressources humaines (éboueurs), des véhicules (camions), des tournées de collecte et des communications citoyennes.

Contexte	Ville d'Antananarivo — 22 quartiers modélisés comme nœuds d'un graphe pondéré. 148 liaisons bidirectionnelles calculées depuis les coordonnées WGS84 réelles. Connexité totale vérifiée par BFS avant chaque calcul Dijkstra.
----------	---

Objectifs fonctionnels

Code	Objectif	Description	Algorithme
OBJ-1	Optimisation des trajets	Calculer le plus court chemin entre deux quartiers	Dijkstra + MinHeap
OBJ-2	Ordonnancement des tournées	Prioriser les quartiers par score combiné	Glouton
OBJ-3	Gestion multi-rôles	3 interfaces : Admin, Éboueur, Citoyen	—
OBJ-4	Réclamations citoyennes	Dépôt, suivi, résolution avec cycle de statut	—
OBJ-5	Notifications temps réel	Polling HTMX toutes les 30 s — fragment HTML	—
OBJ-6	Connectivité du graphe	Vérification avant chaque calcul de chemin	Union-Find
OBJ-7	Application distribuable	Packagée en .dmg / .exe / .ApplImage	electron-builder

Périmètre des utilisateurs

Profil	Volume estimé	Accès	Interface
Administrateur	1–3 agents	Lecture + écriture totale	/admin/*
Éboueur	10–50 agents	Lecture planning + mise à jour statut	/worker/*
Citoyen	Illimité	Lecture + dépôt de réclamation	/citizen/*

02 Architecture du système

L'architecture repose sur un modèle SSR (Server-Side Rendering) hébergé localement par Electron. Le serveur Express tourne dans le main process Electron et génère le HTML complet à chaque requête. HTMX gère les interactions partielles sans rechargement de page, ce qui donne une expérience proche d'une Single Page App sans les complexités d'un framework JavaScript front-end.

Couches applicatives

Couche	Technologie	Responsabilité	Pattern
Présentation	Electron BrowserWindow	Fenêtre native — charge localhost:3131	Desktop Shell
Transport	HTTP / Express	Routage, middleware, sessions	MVC
Rendu	EJS + Partial	Génération HTML côté serveur	SSR
Interactivité	HTMX 1.9	Mise à jour partielle de fragments HTML	Hypermedia
Métier	Controllers	Validation + logique + appel algorithmes	Service Layer
Algorithmes	JS natif	Dijkstra, Glouton, Union-Find — fonctions pures	Pure Functions
Persistance	SQLite / MySQL	Stockage relationnel, mêmes requêtes SQL	DAO / Model

Flux de démarrage Electron

[illegible]

Arborescence complète du projet

```

tana_clean/
■ ■ ■ ■ CLAUDE.md ← Guide de développement
■ ■ ■ ■ package.json ← Scripts npm + dépendances
■
■ ■ ■ ■ electron/
■ ■ ■ ■ main.js ← Main process : démarre Express, crée BrowserWindow
■
■ ■ ■ ■ src/
■ ■ ■ ■ server.js ← Point d'entrée Express (port 3131)
■ ■
■ ■ ■ ■ routes/
■ ■ ■ ■ admin.js ← Routes /admin/* (protégées role=admin)
■ ■ ■ ■ worker.js ← Routes /worker/* (protégées role=worker)
■ ■ ■ ■ citizen.js ← Routes /citizen/* (protégées role=citizen)
■ ■
■ ■ ■ ■ controllers/
■ ■ ■ ■ AdminController.js ← Logique métier admin
■ ■ ■ ■ WorkerController.js ← Logique métier éboueur
■ ■ ■ ■ CitizenController.js ← Logique métier citoyen
■ ■
■ ■ ■ ■ algorithms/
■ ■ ■ ■ dijkstra.js ← Plus court chemin (MinHeap)
■ ■ ■ ■ greedy.js ← Ordonnancement par score
■ ■
■ ■ ■ ■ models/
■ ■ ■ ■ db.js ← Connexion SQLite / MySQL
■ ■ ■ ■ User.js Truck.js Tour.js Complaint.js
■ ■ ■ ■ Notification.js Log.js
■ ■
■ ■ ■ ■ views/ ← Templates EJS
■ ■ ■ ■ layout/base.ejs ← Layout principal (sidebar + header)
■ ■ ■ ■ layout/auth.ejs ← Layout page de connexion
■ ■ ■ ■ admin/ ← dashboard, map, planning, complaints
■ ■ ■ ■ worker/ ← dashboard, schedule
■ ■ ■ ■ citizen/ ← dashboard, complaints
■ ■ ■ ■ partials/ ← pagination, drawer, notifications...
■ ■
■ ■ ■ ■ public/
■ ■ ■ ■ css/tokens.css ← Variables CSS (Atlassian Design Tokens)
■ ■ ■ ■ css/app.css ← Styles globaux, composants
■ ■ ■ ■ js/htmx.min.js ← HTMX 1.9
■
■ ■ ■ ■ database/
■ ■ ■ ■ schema.sql ← DDL complet (9 tables)
■ ■ ■ ■ seed.js ← 22 quartiers + 74 arêtes WGS84 + users
■
■ ■ ■ ■ tests/
■ ■ ■ ■ dijkstra.test.js
■ ■ ■ ■ greedy.test.js
■
■ ■ ■ ■ docs/
■ ■ ■ ■ TanaClean2035_Documentation.pdf

```

03 Stack technique

Le stack a été conçu pour minimiser la complexité tout en respectant les contraintes du projet : développement solo, délai de 6 semaines, déploiement desktop. Pas de framework front-end, pas de build step webpack — le serveur Express génère directement le HTML via EJS.

Technologie	Version	Catégorie	Rôle	Justification du choix
Electron	30+	Shell	App desktop	Empaquette l'app web sans navigateur externe
Node.js	20 LTS	Runtime	Serveur	Même langage côté serveur et algorithmes
Express	4.x	Framework	HTTP + SSR	Léger, pas de build step, middleware riche
EJS	3.x	Templating	Rendu HTML	Syntaxe JS native, partials, layouts
HTMX	1.9	Frontend	Interactivité	UX SPA sans React/Vue, déclaratif HTML
better-sqlite3	9.x	BDD	SQLite dev	Synchrone, zéro config, idéal dev
mysql2	3.x	BDD	MySQL prod	Même schéma SQL, migration transparente
bcrypt	5.x	Sécurité	Mots de passe	Hachage cost=10, standard industrie
express-session	1.x	Auth	Sessions HTTP	Stockage côté serveur, cookie httpOnly
express-validator	7.x	Validation	Formulaires	Chaîne de validateurs déclaratifs
Jest	29.x	Tests	Tests unitaires	Tests des algorithmes purs
electron-builder	24+	Build	Packaging	.dmg / .exe / .AppImage depuis package.json

Design System — Atlassian Design Tokens

L'interface s'appuie sur les tokens CSS de l'Atlassian Design System (ADG), reproduits via des variables CSS dans `tokens.css`. Cela donne un rendu identique aux produits Jira/Confluence : fond N20 (#F4F5F7), cartes blanches avec `shadow-card`, lozenges de statut, tableaux `table`.

Token CSS	Valeur hex	Usage
<code>--color-brand</code>	#0052CC	Boutons primaires, liens, en-têtes de section
<code>--color-brand-subtle</code>	#DEEBFF	Fond de panneaux info, badges
<code>--color-bg</code>	#F4F5F7	Fond de page global — jamais blanc pur
<code>--color-surface</code>	#FFFFFF	Cartes, panneaux, modals
<code>--color-border</code>	#DFE1E6	Bordures de tableaux, inputs, séparateurs
<code>--color-text</code>	#172B4D	Texte principal (N800)
<code>--color-text-subtle</code>	#6B778C	Texte secondaire, labels, métadonnées
<code>--color-success</code>	#00875A	Lozenge 'Complété', statut actif
<code>--color-warning</code>	#FF8B00	Lozenge 'En cours', fill camion > 70 %
<code>--color-danger</code>	#DE350B	Lozenge 'Annulé', réclamations urgentes

Commandes de développement

```
# Installation
npm install # Installe toutes les dépendances
node database/seed.js # Crée la DB SQLite + insère les données de test
# Développement
npm run dev # Express seul avec nodemon (rechargement auto)
npm run electron # Electron + Express (mode dev complet)
# Qualité
npm test # Jest — tests dijkstra.test.js + greedy.test.js
npm run test:coverage # Couverture de code
# Production
npm run build # electron-builder — génère l'installateur
npm run build -- --mac # macOS .dmg uniquement
npm run build -- --win # Windows .exe (NSIS) uniquement
```

04 Base de données

La base de données relationnelle comporte 9 tables. En mode développement, SQLite est utilisé via better-sqlite3 (API synchrone, idéale pour Electron). En production, MySQL prend le relais sans modification du schéma SQL. Le graphe géographique des 22 quartiers est modélisé par deux tables dédiées : districts (nœuds) et district_edges (arêtes pondérées par la distance réelle en kilomètres, calculée depuis les coordonnées WGS84).

Graphe Antananarivo	22 quartiers · 74 paires d'arêtes (148 directions) · distances calculées avec $d = \sqrt{(\Delta lat \times 111)^2 + (\Delta lng \times 105)^2}$ km (latitude -18.9°). Connectivité totale vérifiée par BFS depuis Analakely (nœud 1) — tous les nœuds atteignables.
----------------------------	--

Tables et relations

Table	Lignes seed	Description	Colonnes clés
districts	22	Nœuds du graphe — quartiers d'Antananarivo	id, name, population
district_edges	148	Arêtes pondérées (distance km, bidirectionnel)	from_id, to_id, distance
users	12+	Comptes unifiés : admin / worker / citizen	id, name, email, password, role
workers	5	Profil étendu des éboueurs	id, user_id, district_id
trucks	5	Flotte de camions	id, plate, capacity_t, fill_level, status
tours	20+	Tournées planifiées (lien worker ↔ truck ↔ district)	id, worker_id, truck_id, district_id, scheduled_at, status
complaints	15+	Réclamations déposées par les citoyens	id, citizen_id, district_id, content, is_urgent, status
notifications	30+	Alertes destinées aux citoyens	id, citizen_id, message, read, created_at
logs	auto	Journal de toutes les actions utilisateur	id, user_id, action, details, created_at

Cycles de statut métier

Entité	Champ	Valeurs possibles	Transition
Tour	status	planned / active / done / cancelled	Admin → create (planned) · Éboueur → start (active) → done
Complaint	status	open / processing / closed	Citoyen → submit (open) · Admin → traite (processing) → close
Truck	status	available / in_use / maintenance	Automatique lors de l'assignation d'une tournée
Notification	read	0 (non lue) / 1 (lue)	Polling citoyen → marquage lu

Schéma SQL — tables principales

```
-- Graphe géographique
CREATE TABLE districts (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  population INTEGER DEFAULT 0
);
CREATE TABLE district_edges (
  from_id INTEGER NOT NULL REFERENCES districts(id),
  to_id INTEGER NOT NULL REFERENCES districts(id),
  distance REAL NOT NULL, -- km calculés depuis coordonnées WGS84
  PRIMARY KEY (from_id, to_id)
);
-- Utilisateurs unifiés (3 rôles)
CREATE TABLE users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  password TEXT NOT NULL, -- bcrypt hash, cost factor 10
  role TEXT NOT NULL CHECK(role IN ('admin','worker','citizen')),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
-- Tournées
CREATE TABLE tours (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  worker_id INTEGER REFERENCES users(id),
  truck_id INTEGER REFERENCES trucks(id),
  district_id INTEGER REFERENCES districts(id),
  scheduled_at DATETIME NOT NULL,
  status TEXT DEFAULT 'planned'
  CHECK(status IN ('planned','active','done','cancelled'))
);
-- Réclamations
CREATE TABLE complaints (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  citizen_id INTEGER REFERENCES users(id),
  district_id INTEGER REFERENCES districts(id),
  content TEXT NOT NULL,
  is_urgent INTEGER DEFAULT 0,
  status TEXT DEFAULT 'open'
  CHECK(status IN ('open','processing','closed')),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

05 Profils utilisateurs

L'application distingue trois profils avec des droits et des interfaces entièrement séparés. L'authentification est basée sur les sessions Express : après connexion, le rôle est stocké en session et toutes les routes sont protégées par un middleware `requireAuth` qui vérifie rôle ET session.

Administrateur	Accès complet à toutes les fonctionnalités du système.
Rôle DB	admin
Email démo	admin@tanaclean.mg
Mot de passe	password123
Interface	/admin/*
Capacités	Carte Dijkstra, gestion des tournées/éboueurs/camions, traitement des réclamations, envoi de notifications
CRUD autorisés	Éboueurs (créer, modifier), Camions (créer, supprimer), Tournées (créer, statut), Réclamations (statut)

Éboueur	Vue limitée à son planning du jour et à son camion assigné.
Rôle DB	worker
Email démo	worker1@tanaclean.mg
Mot de passe	password123
Interface	/worker/*
Capacités	Consultation du planning trié par score glouton, démarrage / fin de tournée, mise à jour du niveau de remplissage du camion
Restrictions	Ne peut pas créer de tournée, pas d'accès aux réclamations, pas d'accès aux données des autres éboueurs

Citoyen	Accès en lecture au suivi du camion + dépôt de réclamations.
Rôle DB	citizen
Email démo	citizen1@tanaclean.mg
Mot de passe	password123
Interface	/citizen/*
Capacités	Suivi du camion dans son quartier, dépôt et historique de réclamations, réception de notifications (polling 30 s), appel urgent
Restrictions	Accès uniquement à ses propres réclamations, pas d'accès aux données de planning ou d'autres citoyens

06 Flux métier

Authentification et gestion de session

#	Acteur	Action	Détail technique
1	Utilisateur	Accède à /login	Express sert la page EJS auth/login.ejs
2	Utilisateur	Soumet email + mot de passe	POST /login – body: { email, password }
3	Controller	Recherche l'utilisateur par email	SELECT * FROM users WHERE email = ?
4	Controller	Vérifie le mot de passe	bcrypt.compare(password, hash)
5	Système	Crée la session	req.session.userId = id ; req.session.role = role
6	Système	Redirection selon le rôle	res.redirect('/admin' '/worker' '/citizen')
7	Middleware	Protège toutes les routes	requireAuth vérifie session + rôle à chaque requête
8	Utilisateur	Déconnexion	POST /logout → req.session.destroy() → redirect /login

Cycle de vie d'une réclamation

#	Acteur	Action	Statut résultant	Détail
1	Citoyen	Ouvre /citizen/complaints	open	Formulaire de dépôt : contenu + is_urgent
2	Citoyen	Soumet le formulaire	open	POST /citizen/complaints – INSERT en DB
3	Admin	Consulte /admin/complaints	open	Vue liste avec filtres (statut, quartier, page)
4	Admin	Ouvre le détail (drawer HTMX)	open	GET /admin/complaints/:id – fragment HTML
5	Admin	Prend en charge	En cours	POST /admin/complaints/:id/status { status:'processing' }
6	Admin	Résout	Résolu	POST /admin/complaints/:id/status { status:'closed' }
7	Citoyen	Voit le statut mis à jour	Résolu	GET /citizen/complaints — historique personnel

Planification et exécution d'une tournée

#	Acteur	Action	Détail technique
1	Admin	Accède à /admin/planning	Liste des tournées filtrables + formulaire d'assignation
2	Admin	Assigne éboueur + camion + quartier + date	POST /admin/planning – INSERT tour (status='planned')
3	Système	Crée une notification pour le quartier	INSERT notifications WHERE district_id = tour.district_id
4	Éboueur	Consulte /worker/schedule	Tournées triées par score glouton décroissant
5	Éboueur	Démarre la tournée	POST /worker/tours/:id/start → status = 'active'
6	Citoyen	Reçoit la notification	GET /citizen/notifications (polling HTMX hx-trigger='every 30s')
7	Éboueur	Met à jour le camion	POST /worker/trucks/:id/fill { fill_level: 75 }
8	Éboueur	Termine la tournée	POST /worker/tours/:id/done → status = 'done'

Calcul d'itinéraire (Carte Dijkstra)

#	Acteur	Action	Détail technique
1	Admin	Ouvre /admin/map	Formulaire : select départ (22 quartiers) + select arrivée
2	Admin	Soumet le formulaire	POST /admin/map/route { from_id, to_id }
3	Controller	Vérifie la connectivité	Union-Find : connected(from_id, to_id) → 422 si non connexe
4	Algorithme	Calcule le plus court chemin	dijkstra(graph, from_id) → { dist, prev }
5	Controller	Reconstruit le chemin	reconstructPath(prev, from_id, to_id) → tableau de nœuds
6	Système	Retourne le résultat	JSON { path:[...], distance: 4.7, names:[...] }
7	HTMX	Affiche sans rechargement	hx-post + hx-target='#result' → fragment HTML remplace le bloc

07 Opérations par profil

Administrateur

18 opérations couvrant la totalité du système.

Module	Méthode	Route	Description	HTMX
Dashboard	GET	/admin	Compteurs : éboueurs actifs, camions, réclamations, tournées du jour	Non
Carte	GET	/admin/map	Interface Dijkstra : départ / arrivée	Non
Carte	POST	/admin/map/route	Calcul du plus court chemin — JSON {path, distance, names}	Oui
Planning	GET	/admin/planning	Liste des tournées avec filtres (statut, quartier, date) + pagination	Partiel
Planning	POST	/admin/planning	Créer une tournée : éboueur + camion + quartier + date	Non
Planning	GET	/admin/planning/:id	Détail d'une tournée — fragment drawer	Oui
Planning	POST	/admin/planning/:id/status	Modifier le statut (planned/active/done/cancelled)	Non
Éboueurs	GET	/admin/workers	Liste paginée avec quartier de base	Partiel
Éboueurs	POST	/admin/workers	Créer un compte éboueur avec hachage bcrypt	Non
Éboueurs	GET	/admin/workers/:id	Détail : tournées assignées, statistiques	Oui
Camions	GET	/admin/trucks	Liste avec statut (disponible/en service/maintenance)	Partiel
Camions	POST	/admin/trucks	Créer un camion : plaque + capacité + statut initial	Non
Camions	GET	/admin/trucks/:id	Détail : historique des tournées, fill_level	Oui
Camions	POST	/admin/trucks/:id/delete	Suppression avec dialog de confirmation JS	Non
Réclamations	GET	/admin/complaints	Liste filtrée (statut, quartier) + pagination	Partiel
Réclamations	GET	/admin/complaints/:id	Détail : contenu, citoyen, quartier, historique	Oui
Réclamations	POST	/admin/complaints/:id/status	Cycle open → processing → closed	Non
Notifs	POST	/admin/tours/:id/notify	Insertion en table notifications pour les citoyens du quartier	Non

Éboueur

5 opérations centrées sur l'exécution des tournées.

Opération	Méthode	Route	Description
Dashboard	GET	/worker	Résumé : prochaine tournée, camion assigné, jours de collecte
Planning du jour	GET	/worker/schedule	Tournées triées par score glouton décroissant
Démarrer tournée	POST	/worker/tours/:id/start	status → 'active', truck.status → 'in_use'
Terminer tournée	POST	/worker/tours/:id/done	status → 'done', truck.status → 'available'
Niveau camion	POST	/worker/trucks/:id/fill	Met à jour fill_level (0–100 %) — barre de progression

Citoyen

5 opérations orientées suivi et communication.

Opération	Méthode	Route	Description
Dashboard	GET	/citizen	Statut camion dans son quartier + compteur notifications non lues
Mes réclamations	GET	/citizen/complaints	Formulaire de dépôt + historique filtré sur citizen_id
Déposer réclamation	POST	/citizen/complaints	INSERT complaint avec district_id, content, is_urgent
Notifications	GET	/citizen/notifications	Fragment HTML – hx-trigger='every 30s' – marque read=1
Appel urgent	POST	/citizen/urgent	Enregistre une demande urgente — score glouton = 1000

08 Algorithmes

Les trois algorithmes sont implémentés comme des fonctions pures en JavaScript natif (aucune dépendance, aucun accès base de données). Ils sont testés unitairement avec Jest et appelés exclusivement depuis les controllers.

8.1 Dijkstra — Plus court chemin ($O((V+E) \log V)$)

L'algorithme de Dijkstra calcule le chemin de coût minimal entre un nœud source et tous les autres nœuds du graphe orienté pondéré à poids positifs. L'implémentation utilise un min-heap (tas binaire) pour extraire le nœud de distance minimale en $O(\log V)$ plutôt que $O(V)$ avec un tableau simple.

Propriété	Valeur	Remarque
Complexité temporelle	$O((V + E) \log V)$	Avec min-heap — optimal pour graphe sparse
Complexité spatiale	$O(V + E)$	Tableau dist[] + liste d'adjacence + heap
Condition préalable	Poids ≥ 0	Distances km toujours positives ☐
Nœuds (V)	22 quartiers	Antananarivo
Arêtes (E)	148 directions	74 paires bidirectionnelles
Densité du graphe	Sparse (~6.7 E/V)	Min-heap avantageux
Fichier source	src/algorithms/dijkstra.js	—
Route d'appel	POST /admin/map/route	Retour JSON
Tests	tests/dijkstra.test.js	Cas nominal, non-atteignable, même nœud

Implémentation complète

```
// src/algorithms/dijkstra.js
class MinHeap {
  constructor() { this.heap = []; }
  push(node, dist) {
    this.heap.push({ node, dist });
    this._bubbleUp(this.heap.length - 1);
  }
  pop() {
    if (this.heap.length === 1) return this.heap.pop();
    const top = this.heap[0];
    this.heap[0] = this.heap.pop();
    this._sinkDown(0);
    return top;
  }
  get size() { return this.heap.length; }
  _bubbleUp(i) {
    while (i > 0) {
      const parent = (i - 1) >> 1;
      if (this.heap[parent].dist <= this.heap[i].dist) break;
    }
  }
}
```

```

    [this.heap[parent], this.heap[i]] = [this.heap[i], this.heap[parent]];
    i = parent;
  }
}
_sinkDown(i) {
  const n = this.heap.length;
  while (true) {
    let smallest = i;
    const l = 2*i+1, r = 2*i+2;
    if (l < n && this.heap[l].dist < this.heap[smallest].dist) smallest = l;
    if (r < n && this.heap[r].dist < this.heap[smallest].dist) smallest = r;
    if (smallest === i) break;
    [this.heap[i], this.heap[smallest]] = [this.heap[smallest], this.heap[i]];
    i = smallest;
  }
}
function dijkstra(graph, source) {
  const dist = {}, prev = {}, heap = new MinHeap();
  for (const node of Object.keys(graph)) dist[node] = Infinity;
  dist[source] = 0;
  heap.push(source, 0);
  while (heap.size > 0) {
    const { node: u, dist: d } = heap.pop();
    if (d > dist[u]) continue; // entrée obsolète dans le heap
    for (const { to, weight } of (graph[u] || [])) {
      const alt = dist[u] + weight;
      if (alt < dist[to]) {
        dist[to] = alt;
        prev[to] = u;
        heap.push(to, alt);
      }
    }
  }
  return { dist, prev };
}
function reconstructPath(prev, source, target) {
  const path = [];
  let cur = target;
  while (cur !== undefined) { path.unshift(cur); cur = prev[cur]; }
  return path[0] === source ? path : []; // vide si non atteignable
}
module.exports = { dijkstra, reconstructPath, MinHeap };

```

8.2 Algorithme Glouton — Ordonnancement des tournées ($O(n \log n)$)

Le glouton calcule un score de priorité pour chaque quartier à collecter et les trie par ordre décroissant. C'est une heuristique : elle produit un bon résultat en temps polynomial mais ne garantit pas l'optimalité globale. Elle est idéale pour des décisions en temps réel.

Propriété	Valeur	Remarque
Complexité	$O(n \log n)$	Dominée par le tri <code>Array.sort</code>
Optimalité	Locale (heuristique)	Bonne pratique sur cas réels
Critère 1	Urgence $\times 1\ 000$	Appel urgent $\rightarrow$ score minimum 1 000
Critère 2	Population / 1 000	Favorise les quartiers densément peuplés
Critère 3	Jours sans collecte $\times 10$	Évite l'abandon prolongé d'un quartier
Fichier source	<code>src/algorithms/greedy.js</code>	—
Utilisé par	<code>GET /worker/schedule</code>	Planning affiché à l'éboueur

Formule de score et implémentation

```
// Formule
score(d) = (d.is_urgent ? 1000 : 0)
+ (d.population / 1000)
+ (d.days_since_collection * 10)
// Exemple : quartier urgent, 120 000 hab, 3 jours sans collecte
// score = 1000 + 120 + 30 = 1150  $\rightarrow$  priorité maximale
// src/algorithms/greedy.js
function greedySchedule(districts) {
  if (!Array.isArray(districts) || districts.length === 0) return [];
  return districts
    .map(d => ({
      ...d,
      score: (d.is_urgent ? 1000 : 0)
        + ((d.population || 0) / 1000)
        + ((d.days_since_collection || 0) * 10)
    }))
    .sort((a, b) => b.score - a.score);
}
module.exports = { greedySchedule };
```

8.3 Union-Find (DSU) — Vérification de connectivité ($O(\alpha(n)) \approx O(1)$)

Union-Find (Disjoint Set Union) détecte si deux nœuds appartiennent à la même composante connexe. Il est appelé avant chaque requête Dijkstra pour retourner immédiatement une erreur 422 si la route est impossible, sans exécuter l'algorithme principal.

Propriété	Valeur	Remarque
find()	$O(\alpha(n)) \approx O(1)$	Path compression — α = inverse d'Ackermann
union()	$O(\alpha(n)) \approx O(1)$	Union by rank — arbre presque plat
Build (n nœuds)	$O(n)$	Initialisation parent[i]=i, rank[i]=0
Optimisations	Path compression + Union by rank	Les deux combinées donnent $\alpha(n)$
Usage	Pré-validation Dijkstra	→ 422 si non connexe, sinon exécute Dijkstra

Implémentation avec path compression et union by rank

```
// Union-Find avec path compression + union by rank
class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n + 1 }, (_, i) => i);
    this.rank = new Array(n + 1).fill(0);
  }
  find(x) {
    // Path compression : aplatit l'arbre récursivement
    if (this.parent[x] !== x)
      this.parent[x] = this.find(this.parent[x]);
    return this.parent[x];
  }
  union(x, y) {
    const rx = this.find(x), ry = this.find(y);
    if (rx === ry) return; // déjà dans la même composante
    // Union by rank : attache la racine de rang inférieur
    if (this.rank[rx] < this.rank[ry]) this.parent[rx] = ry;
    else if (this.rank[rx] > this.rank[ry]) this.parent[ry] = rx;
    else { this.parent[ry] = rx; this.rank[rx]++; }
  }
  connected(x, y) {
    return this.find(x) === this.find(y);
  }
}
// Usage dans le controller
// if (!uf.connected(from_id, to_id))
//   return res.status(422).json({ error: 'Quartiers non connexes' });
```

09 Routes HTTP et API

Routes publiques

Méth.	Route	Description	Auth
GET	/login	Page de connexion (EJS)	—
POST	/login	Auth : bcrypt + création session	—
POST	/logout	Destruction session → redirect /login	—
GET	/health	Healthcheck pour le retry-loop Electron	—

Administrateur (role = admin)

Méth.	Route	Description	Auth
GET	/admin	Dashboard — compteurs globaux	admin
GET	/admin/map	Interface carte Dijkstra	admin
POST	/admin/map/route	Calcul chemin — JSON {path, distance}	admin
GET	/admin/planning	Liste tournées + filtres + pagination	admin
POST	/admin/planning	Créer une tournée	admin
GET	/admin/planning/:id	Détail tournée (fragment HTMX)	admin
POST	/admin/planning/:id/status	Modifier statut tournée	admin
GET	/admin/workers	Liste éboueurs paginée	admin
POST	/admin/workers	Créer un éboueur (bcrypt pass)	admin
GET	/admin/workers/:id	Détail éboueur (fragment HTMX)	admin
GET	/admin/trucks	Liste camions paginée	admin
POST	/admin/trucks	Créer un camion	admin
GET	/admin/trucks/:id	Détail camion (fragment HTMX)	admin
POST	/admin/trucks/:id/delete	Supprimer un camion	admin
GET	/admin/complaints	Réclamations avec filtres	admin
GET	/admin/complaints/:id	Détail réclamation (fragment HTMX)	admin
POST	/admin/complaints/:id/status	Changer statut réclamation	admin
POST	/admin/tours/:id/notify	Notifier citoyens du quartier	admin

Éboueur (role = worker)

Méth.	Route	Description	Auth
GET	/worker	Dashboard journée	worker
GET	/worker/schedule	Planning trié par score glouton	worker
POST	/worker/tours/:id/start	Démarrer tournée → status=active	worker
POST	/worker/tours/:id/done	Terminer tournée → status=done	worker
POST	/worker/trucks/:id/fill	Mettre à jour fill_level	worker

Citoyen (role = citizen)

Méth.	Route	Description	Auth
GET	/citizen	Dashboard citoyen	citizen
GET	/citizen/complaints	Mes réclamations + formulaire	citizen
POST	/citizen/complaints	Déposer une réclamation	citizen
GET	/citizen/notifications	Fragment notifs — polling 30 s	citizen
POST	/citizen/urgent	Appel urgent (score = 1000)	citizen

10 Sécurité et déploiement

Mesures de sécurité

Domaine	Mesure	Détail technique
Authentification	bcrypt cost factor 10	Hash en DB, jamais le mot de passe en clair, compare() constant-time
Sessions	express-session	Cookie httpOnly + sameSite=strict, secret fort depuis .env
Autorisation	requireAuth middleware	Vérifie req.session.userId ET req.session.role sur chaque route
Validation	express-validator	Chaîne de validateurs dans les controllers — jamais dans les modèles
Injections SQL	Requêtes paramétrées	better-sqlite3 .prepare().run(params) / mysql2 ? placeholders
Journalisation	Table logs	Chaque action critique : user_id, action, details, created_at
Electron	Isolation contextuelle	nodeIntegration: false, contextIsolation: true (défaut Electron 12+)
XSS	EJS auto-escape	<%= %> échappe HTML par défaut – <%- %> utilisé seulement sur HTML de confiance

Variables d'environnement (.env)

```
# Serveur
PORT=3131
SESSION_SECRET=votre-secret-fort-min-32-caracteres
# Base de données
DB_TYPE=sqlite # sqlite | mysql
DB_PATH=./database/tanaclean.db # SQLite uniquement
# MySQL (production uniquement)
DB_HOST=localhost
DB_PORT=3306
DB_USER=tanaclean
DB_PASSWORD=motdepasse-securise
DB_NAME=tanaclean_prod
# Electron
NODE_ENV=production # désactive les logs de debug
```

Pipeline de build et distribution

Étape	Commande	Résultat
1 · Seed	<code>node database/seed.js</code>	Crée la base SQLite avec données réalistes
2 · Tests	<code>npm test</code>	Jest — dijkstra + greedy — 0 erreur exigé
3 · Build macOS	<code>npm run build -- --mac</code>	<code>dist/TanaClean-2035.dmg</code>
4 · Build Windows	<code>npm run build -- --win</code>	<code>dist/TanaClean-2035-Setup.exe</code>
5 · Build Linux	<code>npm run build -- --linux</code>	<code>dist/TanaClean-2035.AppImage</code>
6 · Toutes cibles	<code>npm run build</code>	Génère les 3 installeurs en parallèle

Comptes de démonstration

Rôle	Email	Mot de passe	Données seed
Administrateur	<code>admin@tanaclean.mg</code>	<code>password123</code>	Accès complet, toutes les données visibles
Éboueur 1	<code>worker1@tanaclean.mg</code>	<code>password123</code>	Assigné à Analakely, 3 tournées planifiées
Éboueur 2	<code>worker2@tanaclean.mg</code>	<code>password123</code>	Assigné à Ambanidia, 2 tournées planifiées
Citoyen 1	<code>citizen1@tanaclean.mg</code>	<code>password123</code>	Quartier Analakely, 2 réclamations, 3 notifs
Citoyen 2	<code>citizen2@tanaclean.mg</code>	<code>password123</code>	Quartier Faravohitra, 1 réclamation urgente

Couches de protection de l'interface admin

Couche	Mécanisme	Fichier	Comportement
1 — Réseau	Bind 127.0.0.1	server.js	Le port 3132 ne répond qu'à localhost — inaccessible depuis le réseau
2 — Route	Middleware express	server.js	app.use('/admin', ...) retourne use-desktop.ejs sur le serveur web
3 — Login	AuthController	AuthController.js	Admin sur port 3131 → pas de session créée → page use-desktop
4 — Route worker/citizen	requireWeb middleware	middleware/auth.js	Sur port 3132 → retourne desktop-only.ejs si non-admin
5 — Session	requireAuth	middleware/auth.js	Vérifie session + rôle sur toutes les routes protégées

Application desktop Electron — détails techniques

Propriété	Valeur	Détail
Port	3132	Défini par process.env.PORT avant require('../src/server')
Bind address	127.0.0.1	BIND_HOST = IS_DESKTOP ? '127.0.0.1' : '0.0.0.0'
Env flag	ELECTRON_APP=1	Positionné dans electron/main.js avant startServer()
Nom de l'app	TanaClean 2035	app.setName() + app.dock.setIcon() macOS
Icône	build/icon.png	PNG 512×512 généré — camion bleu sur fond brand
Titre fenêtre	TanaClean 2035	page-title-updated bloqué — titre fixe
Sécurité webview	nodeIntegration: false	contextIsolation: true — aucun accès Node.js depuis la page web
Retry loop	300 ms × 40 tentatives	Attend que Express soit prêt avant d'ouvrir BrowserWindow

Paramètre «éléments par page» — correction du flux

Le paramètre de pagination stocké dans la table settings est maintenant propagé de façon fiable à toutes les listes via un middleware dédié et la session.

Étape	Composant	Action
1 — Requête	injectPageSize middleware	Lit pageSize depuis req.session.pageSize → DB si absent → stocke en session
2 — Controller	AdminController	Lit res.locals.pageSize et le passe à chaque service : getAll({ page, limit })
3 — Service	Planning/Complaint/Truck /Staff	Accepte { limit } en paramètre optionnel, fallback SettingsService si absent
4 — Sauvegarde	updateSettings	Stocke dans req.session.pageSize ET DB — appliqué dès la requête suivante